Procedural Mistake Detection in Egocentric Video — Status Report

Contents

1. Executive summary . . . 1
2. Problem and contributions . . . 2
3. Method . . . 2
4. Implementation . . . 4
5. Results . . . 7
6. Literature covered . . . 9
7. Threats to validity . . . 10
8. What remains . . . 10
9. Bottom line . . . 11

Target venue: EMNLP 2026 (via ACL ARR)
Benchmark: CaptainCook4D (NeurIPS 2024 Datasets & Benchmarks)
Report date: 26 July 2026
Scope: method, implementation settings, results, and literature coverage.

---

1. Executive summary

We detect procedural mistakes in egocentric cooking video: for each moment of a multi-step recipe, decide whether the person is deviating from the correct procedure and, if so, what kind of mistake it is. Two contributions were proposed. After the completed full-test evaluation, their standing is asymmetric and should be stated plainly:

	Claim	Standing
A	Conditioning a VLM on a procedural task graph enables accurate, interpretable mistake detection	**Demonstrated for binary detection** on the full test split. Mistake *typing* remains weak. The graph's specific contribution is **not yet isolated**.
B	An uncertainty-aware adaptive threshold beats a fixed threshold	**Falsified as stated.** Adaptive loses to fixed in every comparison, in both protocols, for both models.

The headline evidence: on all 109 test recordings under an identical protocol, graph-conditioned fine-tuning lifts binary detection F1 from **0.036 to 0.471** (online). The same evaluation shows the adaptive threshold *reducing* F1 from 0.471 to 0.373. Both findings come from the same run, so neither can be attributed to protocol differences.

Two caveats bound how strongly any of this can be claimed: (i) every number is a single-run point estimate with no confidence intervals, and (ii) the reproduced prior-work baseline still scores marginally higher on the one metric it reports.

---

2. Problem and contributions

Task. Given a first-person video of someone performing a multi-step procedure, decide *online* — window by window, without access to the future — whether the person has deviated from the correct procedure, name the mistake type, and localize when the deviation began.

Contribution A — Procedural Graph-Guided VLM Reasoning. The correct procedure is represented explicitly as a machine-readable task graph: steps plus precedence constraints, with associated objects and expected states. Rather than training a classifier over video features, we condition a vision-language model on this graph at inference time. At each window the model receives the frames *together with* where it should expect the person to be in the procedure — the current step, the set of valid next steps, the required objects, the expected resulting states, and any permitted variations — and answers in a structured form: is this a deviation, of what type, with what confidence, and why. The hypothesis is that the graph supplies a *language-structured prior* that a graph-blind model lacks, yielding both better accuracy and a human-readable rationale. The model is then LoRA fine-tuned on the benchmark so it learns the task's visual vocabulary while the class set stays in the prompt rather than in a fixed classifier head.

Contribution B — Uncertainty-Aware Adaptive Decision Threshold. A window is flagged only when the model reports a deviation *and* its confidence clears a bar τ. The proposal is that τ should not be a constant but should rise and fall per window with interpretable signals: the complexity of the task, how many valid ways the step admits, the model's own uncertainty, how well the scene matches the graph, and how inconsistent the model has been over recent windows. The design intuition is that the bar should rise when the model is unreliable — demand more confidence in complex, ambiguous, or poorly-matched situations — and fall when the signals are clean. The weights are learned from the validation split rather than hand-set.

---

3. Method

3.1 Benchmark selection and the dataset pivot

The project began on **EgoOops** (ICCVW 2025) and moved to **CaptainCook4D** (NeurIPS 2024 D&B). The reason was decisive and worth recording: EgoOops is 50 videos *in total* — 30 train / 10 val / 10 test, 538 segments, 95 mistakes, ~47k frames at ~10% positive rate. At that scale the reproduced prior-work baseline collapsed to an F1 of 0.00 on the test split. A baseline that scores zero is not a baseline, and contributions cannot be demonstrated on data too small to train on. This is a data-scale problem, not a tuning problem.

CaptainCook4D was chosen because it is close to purpose-built for these two contributions:

- it **ships gold task graphs**, one per recipe — precisely the input Contribution A requires, and it allows validating the graph-guided idea against a *correct* graph rather than only an LLM-generated one;
- its error annotations are **temporally bounded**, which is what makes the online causal protocol possible;
- it includes **free-text error descriptions**, supporting later explanation metrics;
- it is **ungated**, so reproducibility is clean;
- it is roughly 14× the frames of EgoOops at about twice the positive rate.

Alternatives were surveyed and rejected: IndEgo (repository not public until ~March 2026) and Ego4D-PMD (synthetic, frame-level, no temporal bounds). EgoOops was subsequently removed from the project entirely.

Dataset facts (audited, not quoted from the paper): 384 recordings across 24 recipes, ~94.5 hours; 5700 step-segments of which 1964 carry at least one error; 2574 error instances (a step may carry several); 281 error segments are non-localizable. Official split: 213 train / 62 val / 109 test recordings. Our evaluation covers 14,129 validation windows and 23,696 test windows with zero recordings dropped and zero graphs missing.

3.2 Taxonomy

The project originally maintained a lossy 8→6 remap from CaptainCook4D's categories onto a generic six-class scheme, with two documented policies (STRICT and LENIENT) reported side by side, including a manual audit of all 410 "Preparation" errors. Once EgoOops was dropped, the reason for the remap disappeared, and we adopted CaptainCook4D's **native taxonomy** minus the `Other` catch-all (8 instances total) — **8 classes**:

`no_mistake`, `order_error`, `technique_error`, `preparation_error`, `measurement_error`, `missing_step`, `timing_error`, `temperature_error`

Steps carrying multiple error tags are reduced by a documented priority order in which structural mistakes (missing, out-of-order) outrank execution mistakes (technique, timing, temperature).

A structural limitation must be reported with every result. `missing_step` errors are exactly the non-localizable annotations: a step that never happened produces no video window to look at. A windowed detector therefore *cannot* emit this class — its test support is 2 windows and its F1 is 0.000 in every row. In practice the method covers **7 of the 8 classes**, and macro-F1 should be reported both with and without `missing_step`.

Class support is also severely imbalanced. On the test split: `no_mistake` 16,584 windows, `order_error` 4212, `technique_error` 846, `measurement_error` 786, `preparation_error` 621, `timing_error` 556, `temperature_error` 89, `missing_step` 2. Per-class F1 for the last two is statistically meaningless.

3.3 Detection method

The video is tiled into short contiguous windows (4 seconds, 1 fps sampling, 4-second stride — so the windows abut rather than overlap), processed strictly in temporal order. Each window's ground truth is the mistake annotation covering its midpoint.

For each window the vision-language model receives the sampled frames plus the graph context for where it is expected to be in the recipe, and a short summary of its own recent judgements. It returns a structured answer: the observed action and objects, the step it believes it is seeing, a deviation flag, a mistake type, a confidence, an uncertainty reason drawn from a fixed vocabulary (occlusion, ambiguous action, valid variation, short clip, weak graph match), and a free-text rationale contrasting expected with observed.

Step tracking differs between the two protocols, and this is the substantive difference between them. Online, the expected step advances by one whenever the model reports no deviation — the graph acts as a state machine the model walks. Offline, with the whole video available, the step index is tied to the window index and consequently saturates at the final step for any recording longer than the graph, so its conditioning is weaker. This is the main reason offline scores below online throughout, and it is a limitation of the current offline treatment rather than a property of offline access.

3.4 Confidence — a substantive methodological finding

The model's *verbalized* confidence turned out to be unusable, in two stages, and correcting this is one of the more transferable findings of the project.

Zero-shot, the verbalized confidence took only two values (0.8 and 0.9) and was statistically identical on true positives (0.859) and false positives (0.863). After fine-tuning it became strictly worse: the training targets hard-coded `confidence = 0.9`, and the model memorized it exactly, emitting a single constant on every window. With constant confidence, *any* threshold rule is inert — adaptive and fixed necessarily make identical decisions.

The fix required no retraining: read confidence from the **generation logits** instead of the generated text — specifically, the model's probability mass on the boolean decision token it emits for `deviation`. This is a genuine per-window signal. On a probe of real test windows it ranged 0.117–0.945 with the correct polarity (low on non-mistakes, high on confident deviations), against the constant 0.900 of the verbalized field.

The defensible claim is therefore: **verbalized VLM confidence is degenerate, and the decision-token probability is a usable substitute.** The claim that is *not* supported is that it is well-calibrated — expected calibration error remains 0.47–0.55 across the final results.

3.5 Adaptive threshold method

τ is a clipped linear function of five interpretable features — task complexity, inverse step-variant count, composite uncertainty, graph-match deficit, and recent-history inconsistency — plus a bias.

An earlier formulation passed this through a sigmoid, which mathematically squashed τ into a narrow band around 0.5; with coarse confidences it never left that band, so adaptive silently collapsed onto fixed. Rewriting τ as a clipped linear bar over the full range fixed this.

The weights are not hand-set. They are fit on validation by logistic regression restricted to windows the model already flagged as deviations (τ only bites there), predicting the true label from confidence plus the five features; the learned decision boundary is then rearranged into the threshold parameters. The fit is required to find a *positive* confidence coefficient — if higher confidence is not associated with being correct, the adaptive rule is ill-posed, and the procedure refuses, falls back to defaults, and records the refusal rather than inventing signal.

3.6 Post-processing and evaluation protocol

Per-window decisions are median-smoothed, and the onset of a deviation is localized as the first run of consecutive fired windows. Every evaluation reports **online (causal) and offline (full-video) side by side**, and within each, **fixed and adaptive τ side by side** — with each variant re-gating its own typed class stream and its own score stream, so the comparison is genuine rather than an artifact of sharing intermediate quantities.

Metrics: binary F1 (mistake vs not), macro-F1 and per-class F1 *reported with support*, AUC-ROC, precision and recall, expected calibration error and Brier score, and — online only, since earliness is a causal notion — an early detection score and the mean absolute error of the predicted onset time.

Several metric-level corrections were made during hardening and are worth noting as due diligence: per-class F1 now scores a never-predicted class as 0.0 rather than silently dropping it; calibration metrics are computed against prediction *correctness* rather than the positive label; and coverage accounting (recordings evaluated, windows, graphs missing, non-localizable events) is surfaced with every run so nothing is silently capped.

3.7 Fine-tuning method

Contribution A's model is LoRA fine-tuned on the benchmark: rank 32, applied to both attention and MLP projections, ~95M trainable parameters (~1.1% of the model). Training is **completion-only language-model cross-entropy** — the model generates the target structured answer, with prompt and image tokens masked out of the loss. This matters for a likely reviewer question: the mistake taxonomy is *not* baked into a classifier head; it lives in the prompt and targets, so the class set can change without retraining the architecture.

Data is a class-balanced ~10k-window subset of the training recordings (roughly 5k mistake, 5k normal), two epochs.

Two training-pipeline problems had to be corrected before any of this ran meaningfully, both of which would have silently wasted GPU time: an eager data loader that would have required ~240 GB of RAM at this dataset's scale, and a loss computed over the entire input sequence (~1850 prompt and image tokens) rather than the ~71 answer tokens, which drowned the training signal and left the loss plateaued around 6.1. After correcting the masking, the loss dropped immediately and converged to ~0.017.

---

4. Implementation

Settings and procedural specifics, for reproducibility and for questions about exactly what was run. Every value below is what produced the results in §5.

4.1 Model, decoding, and hardware

The reasoning model is Qwen2.5-VL-7B-Instruct, run in bfloat16 on a single GPU (an RTX PRO 6000, 96 GB). Decoding is **greedy** — temperature 0, no sampling — so runs are deterministic given the same inputs, with a generation cap of 512 new tokens. The global random seed is 42.

For fine-tuned evaluation the LoRA adapter is merged into the base weights before inference rather than kept as a separate branch, so the fine-tuned and zero-shot runs differ only in weights and are otherwise the same execution path.

4.2 Video sampling and windowing

Frames are decoded directly from the 360p video files. Each window spans **4 seconds** and is sampled at **1 frame per second**, giving roughly four frames per model call, with frame indices taken by striding the native

frame rate.

The window **stride is 4 seconds and was locked at that value for every reported run**. Since the stride equals the window length, the windows are contiguous and **non-overlapping**, not sliding-with-overlap. (The configuration default is a 2-second stride, which would overlap; it was not used for any reported result.)

Windows are emitted while a full window fits inside the recording, so a trailing remainder shorter than one stride is left unsampled — a floor-tiling effect, disclosed in the results footnotes. Test evaluation covers 23,696 windows across 109 recordings; validation covers 14,129 windows across 62.

A window's ground-truth label is the annotation whose time span contains the window's **midpoint**; where several annotations overlap that midpoint, the first match supplies the type.

4.3 Task-graph representation

Each recipe's gold DAG is converted into our internal graph representation: one node per step carrying its description, plus precedence edges. The dataset's START and END sentinels are retained as nodes so edge endpoints stay valid, and marked non-executable. Graph validation rejects dangling edge endpoints at load time. Valid next steps for the current step are read off the outgoing precedence edges.

Twenty-four graphs correspond to the twenty-four recipes and are the ones actually used; the cache also holds five leftover graphs from the abandoned benchmark, which no recording maps to and which are therefore inert. Step counts per recipe range from **9 to 27** (mean ≈ 16.6).

An important honesty point about what the conditioning actually contains. The internal representation supports objects, actions, states, and permitted variations — but the dataset's gold graphs supply **only steps and precedence edges**. For all 24 recipes the object, state, action, and variation lists are empty. The prompt therefore advertises five kinds of graph context while three of them are empty on every window of every run. In practice, "graph conditioning" as evaluated means **step description + valid next steps + step position**, nothing more. This materially narrows what the current Contribution-A result can claim, and it explains why two of the threshold features carry no information (§4.6).

4.4 Prompt construction and structured output

The model is given a fixed instruction template filled with the task name, the current expected step and its description, the valid next steps, the required objects, the expected states, the permitted variations, and a summary of its own last three judgements (each rendered as the observed action, the deviation flag, and the predicted class). The template states all eight class definitions explicitly with one-line glosses — including the distinctions most likely to be confused, such as preparation versus measurement, and timing versus temperature — and instructs the model to answer with JSON only.

The requested answer has nine fields: observed action, observed objects, matched step, deviation flag, mistake class, confidence, uncertainty reason, a one-to-two sentence rationale, and a brief expected-versus-observed delta.

Outputs are parsed defensively. The JSON object is extracted from the generated text; the mistake class is validated against the eight-class vocabulary and the uncertainty reason against its six-value vocabulary, with anything out of vocabulary silently coerced to the safe default; confidence is clamped to [0,1]. If parsing fails entirely, the window falls back to a no-deviation prediction with low confidence and a weak-graph-match uncertainty reason, and the raw text is retained for inspection. No parse failures occurred in the reported runs.

4.5 Confidence extraction

Generation is run with per-step scores retained. To score a window, the generated token sequence is walked until the `deviation` field is reached, and the first subsequent boolean token is located. The softmax probability of that token is taken as the confidence that the window is a mistake — used directly if the model emitted *true*, and complemented if it emitted *false*. If the boolean cannot be located, the fallback is the geometric mean of the emitted tokens' probabilities, which still varies per window. The verbalized confidence field is retained alongside for reference but is not used in any decision.

4.6 Threshold features and fitting

Five features feed τ, in this order: task complexity, a step-variant signal, the composite uncertainty, a graph-match deficit, and recent-history inconsistency.

- **Task complexity** is the step count divided by 20 — note this is *not* normalized to [0,1] and reaches 1.35 for the largest recipe.
- **Variant signal** is the reciprocal of the largest number of permitted variations on any step. Because the gold graphs encode no variations (§4.3), this is **1.0 on every window of every recording**.
- **Composite uncertainty** is 0.6 × (1 − confidence) + 0.4 × a penalty keyed to the model's stated uncertainty reason, capped at 1. The penalties are: none 0.00, valid variation 0.15, short clip 0.20, ambiguous action 0.25, occlusion 0.30, weak graph match 0.40.
- **Graph-match deficit** is one minus a graph-confidence score that is **hardcoded to 0.8 throughout evaluation**, making this feature a constant 0.2 on every window.
- **History inconsistency** is the fraction of consecutive disagreements in the deviation flag across the previous three windows.

So **two of the five features are constants in every reported run**, and contribute only a fixed offset absorbed into the bias. Only three features carry per-window information.

τ is the clipped linear combination of these five with a bias, restricted to [0,1]. Fitting proceeds on the validation split: windows the model flagged as deviations are selected (τ has no effect elsewhere), and a class-balanced logistic regression predicts the true label from confidence plus the five features. The learned boundary is rearranged into threshold parameters by dividing the negated feature weights by the confidence weight. The procedure requires a strictly positive confidence weight; otherwise the rule is ill-posed and the fit is refused, defaults are written, and the refusal and its reason are recorded rather than silently absorbed.

Outcomes: for the fine-tuned model the fit succeeded with a confidence weight of **+1.447**, producing complexity −0.405, variants +0.447, uncertainty +0.868, graph deficit +0.089, history +0.525, bias +0.481. For the zero-shot model the fit **refused**, with a confidence weight of −0.057, so that row runs on default parameters.

4.7 Decision rules and post-processing

A window fires when the model reports a deviation *and* its confidence reaches the threshold — τ per window for the adaptive variant, a constant 0.5 for the fixed variant. Each variant carries its own score stream, in which a fired window keeps its confidence and a rejected window is zeroed, and its own typed class stream, in which a rejected window is relabelled as no-mistake regardless of what the model called it. Keeping the two streams separate is what makes the typed comparison between fixed and adaptive genuine rather than an artifact of shared intermediates.

Score streams are median-smoothed with a window of 3 (edge-padded). Binary metrics then treat any positive smoothed score as a firing.

Onset localization uses a **different and deliberately stricter rule**: the predicted start is the timestamp of the first run of **at least two consecutive** fired windows, compared against the earliest *localizable* gold event on that recording. A recording whose event never fires scores zero earliness and is excluded from the onset-error average, with the detected count reported so the denominator is auditable. This difference in decision rule between the binary and localization metrics is intentional and is footnoted in the results.

4.8 Step tracking

Online, the expected step starts at the first node and advances by one whenever the model reports no deviation, clamped at the last step — the graph is walked as a state machine driven by the model's own judgements, and a persistent deviation holds position. Offline, the expected step is tied to the window index, so it saturates at the final step for any recording with more windows than the graph has steps, which is the common case. This is the substantive difference between the two protocols and the main reason offline underperforms online.

4.9 Fine-tuning configuration

LoRA of rank 32 with scaling 64 and dropout 0.05, no bias adaptation, applied to both attention projections (query, key, value, output) and MLP projections (gate, up, down) — about 95M trainable parameters, 1.13%

of the model, as a causal language-modelling task. Learning rate 2e-4 with a cosine schedule and warmup, gradient accumulation 8, two epochs.

Training data is a class-balanced subset of the training recordings drawn at the same 4-second window length, 4-second stride, and 1 fps as evaluation: one negative per positive, capped at 10,000 windows, yielding 4,951 mistake and 5,049 normal windows. The window index is built without decoding pixels — only time bounds and labels — and frames are decoded lazily per item, which is what makes the dataset's ~90k candidate windows tractable.

The loss is computed **only over the answer tokens**, with prompt and image tokens masked out. Training ran about 3 hours and converged to a final loss of ≈0.017. The resulting adapter is 364 MB.

One consequential detail of the training targets: the `confidence` field in the supervision was a **hardcoded constant**, which the model memorized exactly. This is the direct cause of the degenerate verbalized confidence described in §3.4 and the reason confidence had to be read from the logits instead.

4.10 Evaluation harness and runtime

The full ablation runs as four sequential passes on a single GPU: collect fine-tuned predictions over the whole validation split and fit its threshold; the same for the zero-shot model; then the zero-shot model over all 109 test recordings under both protocols; then the fine-tuned model likewise. Fixed and adaptive variants are computed within a single pass rather than by re-running the model, since both are gating decisions over the same predictions. Total runtime was roughly 38 hours; the run completed on 20 June 2026.

The validation collections were deliberately regenerated from scratch rather than reused, because earlier validation predictions predated the logit-based confidence change and carried the degenerate constant.

Each run records coverage explicitly — recordings evaluated, windows processed, graphs missing, frames dropped, localizable events — so that nothing is silently capped. All reported runs show zero recordings skipped, zero graphs missing, and zero dropped frames.

Metric computation fixes worth recording: per-class F1 is computed against an explicit label list so a class the model never predicts scores 0.0 instead of disappearing from the average; calibration error and Brier score are computed against prediction *correctness* rather than against the positive class; and support counts accompany every per-class figure.

A 48-case unit-test suite covers the taxonomy, graph schema, windowing, threshold behaviour, and metrics, and runs without a GPU or the dataset.

4.11 Implementation caveats carried into the results

Five items in this section limit interpretation and are repeated in §7 so they are not lost:

1. Graph conditioning is **steps and precedence only** — objects, states, and variations are empty in the gold graphs (§4.3).
2. Two of the five threshold features are **constant** in every run (§4.6).
3. The complexity feature is an unnormalized proxy that can exceed 1, and its learned weight came out negative.
4. Offline step conditioning **saturates** for most recordings (§4.8).
5. Binary metrics and onset localization use **different firing rules** by design (§4.7).

---

5. Results

5.1 Reproduced baseline — MistSense (ICCV 2025)

MistSense was reproduced end-to-end and retrained on CaptainCook4D so that a comparison exists on the *same* data under the *same* protocol.

Metric	On EgoOops (collapsed)	On CaptainCook4D
f1_mistake, per-window (872 windows)	0.00	**0.495** @ τ=0.70

Metric	On EgoOops (collapsed)	On CaptainCook4D
f1_mistake, online per-frame (189,885 frames)	0.031	0.452 @ τ=0.70
overall F1, per-window	~0	0.547
precision / recall	—	0.31 / 0.82

The move from 0.00 to ~0.49 is attributable purely to data scale — same architecture, ~14× the frames, ~2× the positive rate. For context the original paper reports F1 63.3 on Epic-Tent-O and 55.7 on HoloAssist, so ~0.49 on a different dataset is the same ballpark and the baseline is credible rather than crippled.

Two honesty notes carried forward to the paper: the reproduction substitutes an ungated LLM and a CLIP visual encoder for the paper's gated components (this affects only the captioning stages, not the detector we compare against); and the baseline over-fires (recall ~0.82, precision ~0.31), a known weakness we deliberately did not tune away, since polishing a baseline is not where the contribution lies.

5.2 Full test ablation — all 109 test recordings

This is the definitive result set: every test recording, 23,696 windows, no subsetting, both protocols, both threshold variants.

Online (causal)

System	binary F1	macro F1	AUC	P	R	ECE	EDS	onset MAE (s)
Zero-shot + fixed τ	0.036	0.107	0.497	0.259	0.019	0.549	0.245	236.2
Zero-shot + adaptive τ	0.017	0.104	0.497	0.212	0.009	0.548	0.115	317.6
LoRA + fixed τ	**0.471**	**0.185**	**0.616**	0.408	0.556	0.502	**0.763**	**60.2**
LoRA + adaptive τ	0.373	0.170	0.580	0.451	0.318	0.472	0.369	102.1
MistSense baseline	0.495	—	—	—	—	—	—	—

Offline (full-video)

System	binary F1	macro F1	AUC	P	R
Zero-shot + fixed τ	0.040	0.109	0.498	0.281	0.022
Zero-shot + adaptive τ	0.022	0.106	0.499	0.288	0.012
LoRA + fixed τ	**0.445**	**0.189**	0.601	0.424	0.469
LoRA + adaptive τ	0.297	0.166	0.558	0.469	0.218

Per-class F1 for the best row (LoRA + fixed, online): `no_mistake` 0.705, `order_error` 0.326, `technique_error` 0.167, `timing_error` 0.151, `measurement_error` 0.074, `preparation_error` 0.058, `temperature_error` 0.000, `missing_step` 0.000. Localization: 55 of 61 localizable events detected (adaptive: 45 of 61).

5.3 What the results establish

Contribution A works, and the effect is large. Graph-conditioned fine-tuning moves binary F1 from 0.036 to 0.471 online and 0.040 to 0.445 offline, with AUC moving from chance (0.497) to 0.616. Zero-shot Qwen2.5-VL is effectively a non-detector on this task — it fires on under 2% of true mistake windows and its ranking is indistinguishable from chance. Because both rows come from the same recordings, windows, and protocol, this is a like-for-like comparison. It supersedes all earlier subset-based figures, which should not be cited.

Contribution B is falsified, not merely unproven. Adaptive τ loses to a fixed τ in all four comparisons — by 0.098 F1 for the fine-tuned model online, 0.148 offline, and on both zero-shot rows. The mechanism is not at fault: the fit for the fine-tuned model was genuine, with a strongly positive confidence coefficient (+1.447). What the adaptive rule actually does is trade a large recall loss (0.556 → 0.318) for a small precision gain (0.408 → 0.451), and it halves earliness (early detection 0.763 → 0.369, onset error 60 s → 102 s). For the zero-shot model the fit refused outright (negative confidence coefficient) and fell back to defaults. Prior reports left this as "an open question pending the test run"; the test run has answered it.

The learned weights contradict the design intuition. The fitted complexity weight is **negative** (−0.405): the data says more complex tasks should get a *lower* bar, the opposite of the stated design. This must be explained or the feature re-examined; it should not be hidden. One plausible reading is that the complexity proxy is really tracking recording length rather than difficulty.

Mistake typing is weak. Macro-F1 peaks at 0.185. The system separates mistake from non-mistake and labels ordering errors moderately (0.326), but largely cannot name the finer types, and two classes are unreachable or unsupported. Either the claim is reframed toward binary detection with partial typing, or typing needs dedicated work.

The baseline still edges us on the one metric it reports. MistSense 0.495 vs our best 0.471. The comparison is not clean — MistSense is binary-only with no typing, runs at a different frame rate and window definition, and its threshold was selected by a post-hoc sweep on the test data (effectively an oracle) — but the honest framing today is *competitive with, not better than*, and the matched re-run is still owed.

Calibration is poor throughout (ECE 0.47–0.55, Brier 0.49–0.52). The token-probability confidence is informative but uncalibrated.

5.4 Statistical validity — the largest gap

Every number above is a **single-run point estimate**: one seed, no confidence intervals, no significance testing. The zero-shot-to-fine-tuned gap is large enough that it is almost certainly real; the fixed-versus-adaptive gap is similarly large but in the *unfavourable* direction. Neither has been tested. Per-recording bootstrap intervals and paired tests on window decisions are the immediate next step and are required before submission.

---

6. Literature covered

This section is deliberately explicit about *depth* of engagement, because the distinction matters for what we can currently write.

6.1 Engaged in depth (reproduced or used as primary infrastructure)

MistSense — Patsch, Wu, Zakour, Salihu, Steinbach, ICCV 2025. *"Versatile Online Detection of Procedural and Execution Mistakes."* Our baseline, and the only prior method we engaged with at implementation depth. A three-stage design: a detection head over RGB and hand features trained with weighted binary cross-entropy; a projection layer feeding an LLM; and LoRA fine-tuning for mistake captioning. We compare against Stage 1, the detector. Reproduced end-to-end from the paper with a documented deviation log, retrained on CaptainCook4D, and evaluated under our protocol. Chosen because it is the closest published method to our setting — online procedural error detection in egocentric video — and recent enough that comparing against it is meaningful for a 2026 submission. Its own reported results (Epic-Tent-O 63.3, HoloAssist 55.7, RGB+Hand) were used to sanity-check that our retrained version is credible.

CaptainCook4D — NeurIPS 2024 Datasets & Benchmarks (arXiv 2312.14556). Primary benchmark. Engaged well past citation depth: we independently audited its schema and counts, converted its gold task graphs into our graph representation, adopted its native error taxonomy, and documented its quirks (recording-to-activity mapping, non-localizable errors, multi-tag steps, the `Other` catch-all).

EgoOops — ICCVW 2025. Original benchmark, empirically evaluated and then rejected on scale grounds after the baseline collapsed on it (documented in §3.1). Retained in the write-up as the justification for the benchmark pivot.

6.2 Surveyed for benchmark selection

IndEgo — rejected: repository not public until ~March 2026. **Ego4D-PMD** — rejected: synthetic, frame-level, without the temporal bounds the online protocol requires. **Epic-Tent-O** and **HoloAssist** — the datasets MistSense reports on; both gated, hence not used directly, but their published numbers anchor our baseline-credibility check. **EgoPER** — noted early as a candidate for cross-dataset evaluation; not pursued.

6.3 Positioned against but not yet engaged in depth

The original plan positions the work against **PREGO**, **MoniTor**, and **ASK-HINT** — the first two as binary online detectors that our multi-class, onset-localizing formulation is meant to extend, and the third as generic-prompt VLM reasoning that our graph grounding is meant to improve on. These are currently **cite-only**: the planning documents assert the contrast, but none has been read closely enough to defend the comparison, and none has been run. MoniTor was at one point planned as a second reproduced baseline and that was not carried out.

6.4 Not yet covered — the honest gap

There is no related-work section, and no literature survey exists for either contribution's methodological neighbourhood. Specifically:

- **Graph-structured or symbolic priors for video reasoning** — the intellectual neighbourhood of Contribution A. We can currently show that fine-tuning helps; we have neither the citations nor the ablation to argue that *graph conditioning specifically* is the mechanism.
- **Adaptive, calibrated, and conformal thresholding** — the neighbourhood of Contribution B. Learning τ by logistic regression over interpretable features has not been justified against the obvious alternatives: Platt scaling, temperature scaling, conformal prediction, or simple per-class thresholds. Given that B is currently falsified in its stated form, this reading is now *more* urgent, not less — the most likely salvage reframes B as a calibration contribution, and that reframing cannot be written without the literature.
- **Confidence extraction from generative models** — our token-probability finding (§3.4) is presented as our own observation. Reading logit-based confidence and verbalized-confidence-reliability work is needed to establish whether it is a contribution or a known result applied to a new setting.

This is the single largest non-experimental gap in the project.

---

7. Threats to validity

1. **No statistical layer.** Single run, one seed, no intervals, no significance tests (§5.4).
2. **The core Contribution-A claim is not isolated.** No graph-on versus graph-off ablation exists, so the reported gain conflates fine-tuning with graph conditioning. This is the most important missing experiment.
3. **Contribution B is falsified in its stated form**, and its learned complexity weight contradicts its design rationale.
4. **Typing is weak** (macro-F1 0.185), with two classes at zero.
5. **`missing_step` is structurally unreachable** by a windowed detector — a limitation of the formulation, to be reported rather than worked around.
6. **Baseline comparability** — binary-only, different frame rate and windowing, oracle-selected threshold; and it still scores marginally higher.
7. **Calibration remains poor** in absolute terms.
8. **Two threshold features are effectively inert** — the graph-match signal is held constant at 0.8 throughout evaluation, and the gold graphs encode no step variants, so both terms reduce to a fixed offset absorbed by the bias (§4.6).
9. **The graph conditioning is thinner than the design describes.** The gold graphs supply only steps and precedence edges; objects, states, and variations are empty on every window of every run (§4.3). Contribution A as evaluated is conditioned on step description, valid next steps, and step position — not on the richer structure the method section proposes. This should be stated directly rather than left for a reviewer to discover, and it raises the priority of the graph ablation.
10. **No related-work positioning** for either contribution (§6.4).

---

8. What remains

In priority order:

1. **Graph-on / graph-off ablation** — the direct test of Contribution A's actual claim. Highest priority; without it the headline result supports "fine-tuning helps," not "the graph helps."
2. **Decide Contribution B's fate.** The current framing cannot be submitted. Options: reframe as calibration of decision-token confidence; move to per-class thresholds; or report it as a documented negative result with the weight-sign analysis. Requires the threshold literature first.
3. **Statistical layer** — bootstrap intervals and paired significance tests for both the zero-shot-versus-fine-tuned and fixed-versus-adaptive comparisons.
4. **Literature survey and related-work section** for both contributions (§6.4), plus close reading of the methods currently cited by assertion only.
5. **Matched baseline re-run** under our frame rate, windowing, and a non-oracle threshold — or a prominent footnote.
6. **Repair or remove the inert threshold features**, and re-examine the complexity proxy given its negative learned weight.
7. **Improve typing, or narrow the claim** to binary detection with partial typing.
8. Lower priority: gold-versus-LLM-generated graph ablation; the baseline's captioning stages; strengthening the offline protocol's step conditioning.

---

9. Bottom line

The system is built, runs end to end on the full benchmark, and produces a large, clean, like-for-like result for Contribution A's binary detection — 0.036 to 0.471 F1 — alongside a genuine methodological finding about VLM confidence. Contribution B, tested properly at full scale, does not work as proposed and needs reframing or retirement. The two things standing between the current state and a submittable paper are experimental (the graph ablation and a statistical layer) and scholarly (a related-work foundation that does not yet exist for either contribution).